Specification of

Device Automation Service

v. 0.0.1.0

Implemented by:

Thomas Pollerspöck

Nguyen Huynh Tri Cuong
Tran Duy Ngoan
Mai Dinh Nam Son
Hua Van Thong
Holger Queckenstedt

03/2025

Contents

Chapter 1

Introduction

The Robot Framework is an automation framework that can be used to execute automated software tests. It's open source, keyword driven, and extensible. Tests are implemented in simple text files.

The extensibility of the Robot Framework gives every test developer the possibility to add missing functionality easily. But when thinking about missing functionality, some aspects urgently need to be considered:

- Additions should be designed carefully to make them as much generic as possible. This will make the additions useful for a wide range of test developers.
- Additions have to be bundled in a meaningful way, and they have to be set under version control.
- Tests need to be reproduced, and therefore also the entire test system (Robot Framework together with all additions) must be reproducible.

And these deliberations are not only related to the Robot Framework itself. Developing software in an efficient way also requires a development environment. And this development environment most probably needs to be configured - at least to be able to handle the additions also. It has to be ensured to use open source software only, to avoid any license fee. And finally it must be ensured that every test developer within a team of developers uses the same development environment with the same settings.

To cover all aspects mentioned above, the following things have been done:

- Download a certain Python version (because the Robot Framework is a Python based application).
- Add further useful Python modules to the Python installation.
- Download a certain version of the Robot Framework. This is the base.
- Implement (carefully) some changes within the core source code of the Robot Framework.
- Add further components to provide useful features that are currently missing in Robot Framework (like a management of test configuration values and a version control mechanism).
- Select a development environment (here: VSCodium).
- Configure this development environment to handle especially the Robot Framework code together with all additions.
- Put all things together in a separate installer.

The outcome is one single installer, that installs all together: the framework, additional libraries, the development environment and all settings.

This is what we call "**RobotFramework AIO**", with **AIO** means: **A**ll **I**n **O**ne.

With this solution the entire test setup is reproducible by every developer on every computer.

And there is no need any more for any user, to install all affected components manually, and there is also no need any more for any user, to spend effort on the configuration of the test setup. After using the RobotFramework AIO installer, test developers immediately can start to implement tests.

Chapter 2

Installation and start

The **RobotFramework AIO** can be installed under Windows and under Linux in this way:

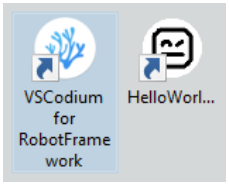
- **Win10**
  Double-click the `setup_.*.exe` file and follow the instructions.
- **Linux**
  If already installed, remove the previous version (this will not remove your own data):

  ```
  sudo apt-get remove robotframework-aio
  ```

  Install/Update now the current version (assumed you are in the download directory):

  ```
  sudo apt-get install ./setup_RobotFramework_AIO_OSD6Linux\_*.deb
  ```

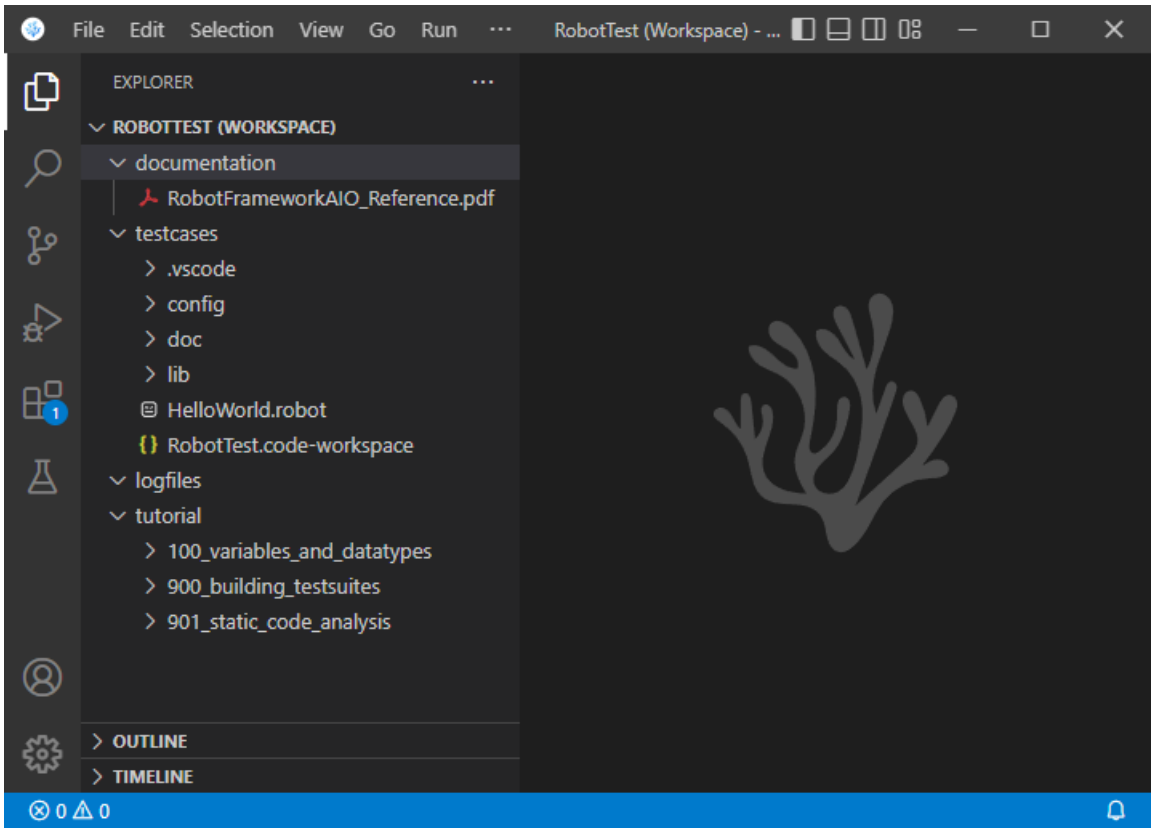
After the installation is finished, a new **VSCodium** icon is present on desktop:

- **Win10**
  

  (together with a link to the example robot file `HelloWorld.robot`)
- **Linux**
  

A double-click starts the application.

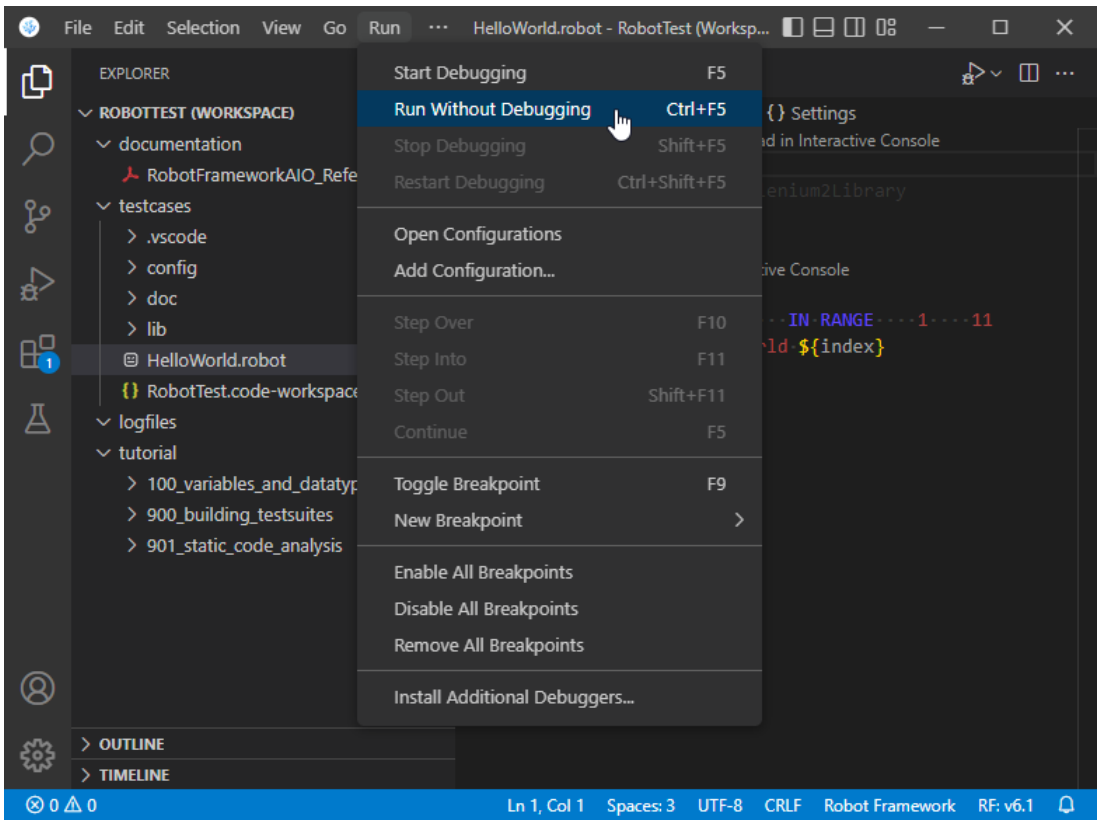
On first start the **VSCodium** main window appears like this:



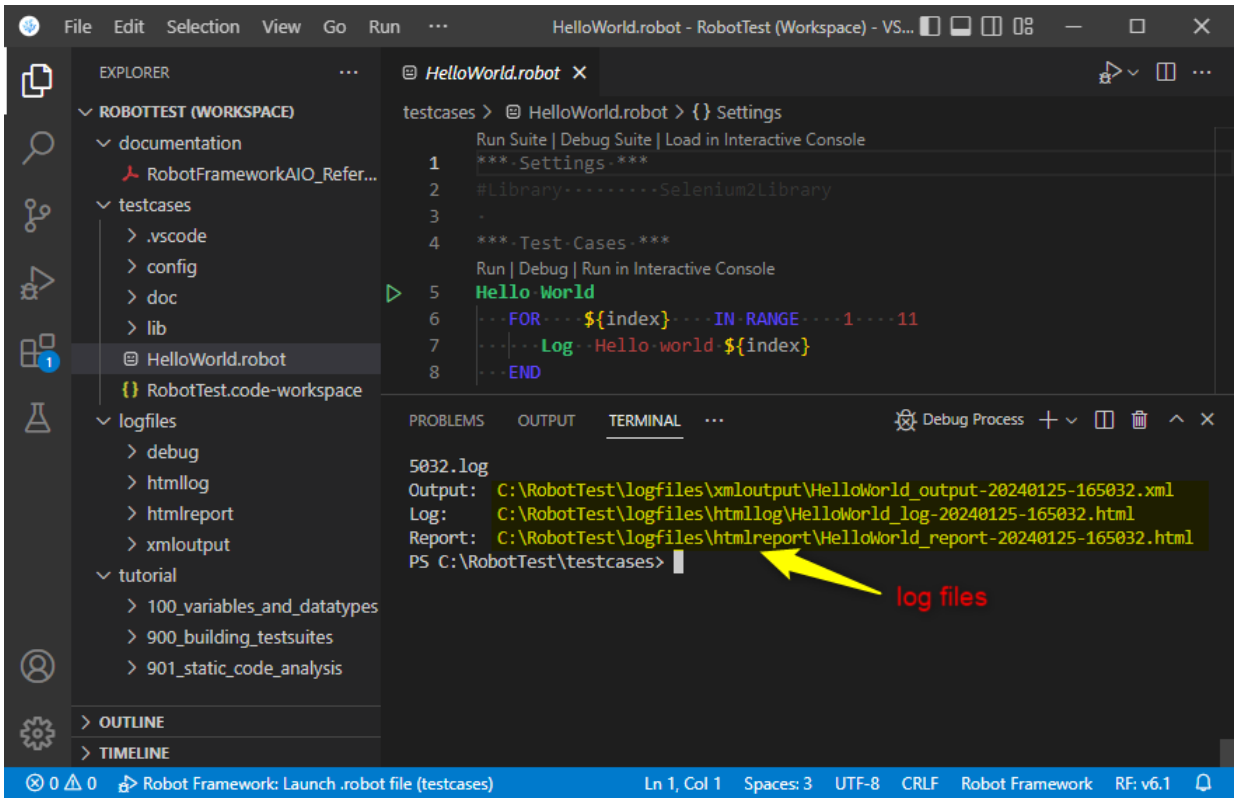
The preconfigured workspace contains the following parts:

- The main documentation `RobotFrameworkAIO_Reference.pdf` (including the documentation of all components that are part of the **RobotFramework AIO**)
- A `testcases` folder containing (beneath other elements) the example robot file `HelloWorld.robot`
- A `logfiles` folder in which all log files and reports will be placed (from tests that are executed with **VSCodium**)
- A `tutorial` folder containing a large bunch of testfiles that can be used for self-dependent learning

For the usage of the predefined infrastructure (e.g. the `logfiles` folder), the **VSCodium** provides an adapted configuration of the **Run** button. This button executes the robot file selected in project explorer of **VSCodium**.



After test execution, the links to log files and reports can be found in the **TERMINAL** window.



Every file there can be opened by CTRL+click on the selected file.

Chapter 3

Components

Compared with the core Robot Framework, the RobotFramework AIO contains some additional components, that extend the functionality by useful features. All components are hosted on GitHub. Most of the components can also be used stand-alone (= outside the context of RobotFramework AIO). But in case of a stand-alone usage, the user is responsible for the installation and configuration. Further hints can be found in the readme files of the corresponding GitHub repository.

The following is an overview about all additional components, that are part of the RobotFramework AIO:

1. RobotFramework_TestsuitesManagement

The **RobotFramework_TestsuitesManagement** enables users to define dynamic configuration values within separate configuration files in JSON format. These configuration values are available during test execution - but under certain conditions that can be defined by the user (e.g. to realize a variant handling or to define parameters that are test bench specific).

The **RobotFramework_TestsuitesManagement** also provides a version control mechanism to ensure that the developed tests fit to the test environment.

Homepage: RobotFramework_TestsuitesManagement

2. JsonPreprocessor

The **JsonPreprocessor** provides additional features in JSON files. These features extend the standard JSON format and make it easier to use JSON files for e.g. configuring tests (see also **RobotFramework_TestsuitesManagement**). The additional features are:

- Add comments
- Let a JSON file import other JSON files (nested imports; e.g. to avoid redundancy)
- Allow also pure Python keywords like `True`, `False` and `None`
- Define new parameters and overwrite already existing ones

Homepage: JsonPreprocessor

3. QConnectBase

QConnectBase is a connection testing library for the Robot Framework. It provides a mechanism to handle trace log continously receiving from a connection (such as Raw TCP, SSH, Serial, etc.) besides sending data back to the other side. It's especially efficient for monitoring the overflowed response trace log from asynchronous trace systems.

Homepage: QConnectBase

4. RobotLog2RQM

RobotLog2RQM writes Robot Framework test results (available in XML format) to the IBM® Rational® Quality Manager (RQM). This includes:

- Create all required resources (Test Case Excution Record, Test Case Execution Result, ...) for new testcase on RQM
- Link all testcases to provided testplan

- Add new test result for existing testcase on RQM
- Update existing testcase on RQM

Homepage: RobotLog2RQM

5. RobotLog2DB

RobotLog2DB writes Robot Framework test results (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: RobotLog2DB

6. PyTestLog2DB

PyTestLog2DB writes test results of the Python module pytest (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: PyTestLog2DB

7. TestResultWebApp

TestResultWebApp is a web-based application and is used for visualizing and tracking test execution results. It provides charts from an overview of the test result to the detail of all included test cases. It also provides tools to control the quality of developing software version (under testing) by the a graphical comparison of test results from different test executions

Homepage: TestResultWebApp

8. GenPackageDoc

GenPackageDoc provides a toolchain to generate a documentation in PDF format out of Python sources that are stored within a repository. The content of this documentation is taken out of the docstrings of functions, classes and their methods. All docstrings have to be written in reStructuredText (RST) format, that is a certain markdown dialect. It is possible to extend the documentation by the content of additional files either in RST format or in LaTeX format. **GenPackageDoc** is also designed to consider setup informations of a repository.

Homepage: GenPackageDoc

9. PythonExtensionsCollection

The **PythonExtensionsCollection** extends the functionality of Python by some useful functions that are not available in Python immediately. This covers for example file and folder operations, string operations like normalizing a path and a pretty print method. Also a file comparison feature is available.

Homepage: PythonExtensionsCollection

10. RobotframeworkExtensions

The **RobotframeworkExtensions** extend the functionality of the Robot Framework by some useful keywords. This covers for example string operations like normalizing a path and a pretty print keyword (especially for composite Python data types). The **RobotframeworkExtensions** keywords are implemented in Python (see **PythonExtensionsCollection**).

Homepage: RobotframeworkExtensions

11. Tutorial

The RobotFramework AIO tutorial contains robot code examples together with the documentation explaining how to use these examples and how they work. The basic idea behind this tutorial is not to have theory only but additionally to this theory also the possibility to experience what is explained inside. The main focus of this tutorial is on some feature extensions available for the Robot Framework.

Homepage: Tutorial

12. Development environment

The RobotFramework AIO installer also installs VSCodium, that is an open source editor. VSCodium is preconfigured especially for the usage together with the RobotFramework AIO.

Features:

- Workspace with included testcases folder, documentation and tutorial
- Previews for certain formats like, MD, RST, PDF, PlantUML
- Support of the extended JSON syntax of **RobotFramework_TestsuitesManagement**
- Static code analysis by Robocop, that is an also included static code checker

Chapter 4

Environment

During an installation of the RobotFramework AIO all content is placed at two different main locations:

- `RobotFramework`
  Contains the framework installation files itself
- `RobotTest`
  Contains the *playground* for the user, including default folder for testcases and logfiles

The root path to these folders can be set by the user during the installation process (called `<root>` in listings below).

The following overview contains a list of environment variables that are introduced by the installer. They can be used to refer to locations inside these folders.

- `RobotLogPath`
  Log files folder in which the RobotFramework AIO places the log files per default.
  Location: `<root>/RobotTest/logfiles`
- `RobotTestPath`
  Folder in which the user can place the testcases.
  Location: `<root>/RobotTest/testcases`
- `RobotTutorialPath`
  Folder containing a tutorial of the RobotFramework AIO (useful for self learning).
  Location: `<root>/RobotTest/tutorial`
- `RobotPythonPath`
  Folder containing the Python installation.
  Location: `<root>/RobotFramework/python39`
- `RobotScriptPath`
  Folder containing scripts of the Python installation.
  Location: `<root>/RobotFramework/python39/scripts`
- `RobotVsCode`
  Folder containing the installation of *Visual Studio Code* that is intended to be the main development environment for the RobotFramework AIO.
  Location: `<root>/RobotFramework/robotvscode`
- `RobotToolsPath`
  Tools which require and extend functionality of the Robot Framework.
  Location: `<root>/RobotFramework/tools`
- `RobotDevtools`
  Tools which are independent of Robot Framework, but are bundeled and tested with RobotFramework AIO.
  Location: `<root>/RobotFramework/devtools`

- `RobotAppium`
  Location of Appium tested and distributed with RobotFramework AIO. More information about Appium you find here: appium
  Location: `<root>/RobotFramework/devtools/Windows/Appium`

- `RobotNodeJS`
  Root folder of `nodjs` tested and distributed with RobotFramework AIO. More information about `nodjs` you you find here: nodejs.org
  Location: `<root>/RobotFramework/devtools/Windows/nodejs`

Chapter 5

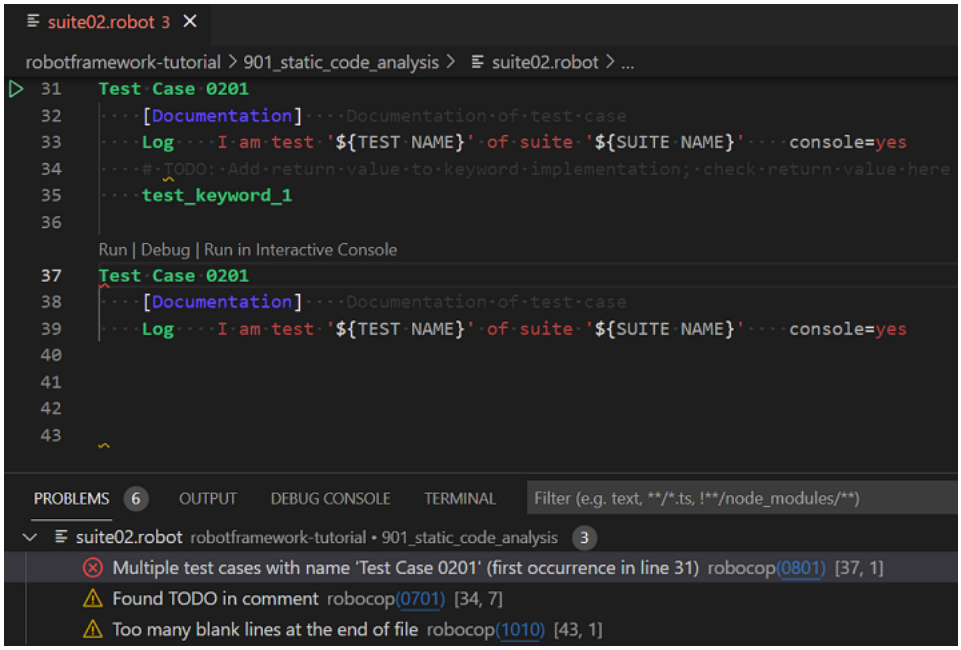
Code analysis

The RobotFramework AIO installation provides a static code analyser to detect potential errors and violations to coding conventions. The name of the analyser is **Robocop**.

Robocop is integrated in Visual Studio Code and and is triggered automatically in case of

1. a file is opened in editor,
2. a file is changed and saved.

The outcome will look like this:



Further examples can be found in the RobotFramework AIO tutorial, chapter `901_static_code_analysis` .

How to configure?

Robocop contains a set of rules that are used to check the source files of the RobotFramework AIO. Not all of them are really useful and some of them are also against our company internal development rules. Therefore we have the need to exclude rules from Robocop execution.

Some rules can be configured (e.g. the rule checking the maximum number of characters within a line of code). Also here we have the need for careful adaptions.

When Robocop is triggered automatically within Visual Studio Code, then the configuration is done with the help of a `.robocop` file. This file has to be placed within the projects folder (see tutorial). To make Robocop results comparable this configuration file should not be modified - or at least should be kept aligned with the version all other project team members use.

Command line

In addition to the automated triggering within Visual Studio Code Robocop can also be called in command line. This is useful

1. in case of a complete folder has to be checked in one step (to avoid the need to open every single source file inside separately),
2. in case of an alternative configuration is wanted (temporarily; to avoid to manipulate the standard configuration file),
3. in case of Robocop shall be triggered by automats like Jenkins,
4. in case of the Robocop results are needed within a log file (output to a log file needs to be configured separately).

To give it a try make a copy of the configuration file `.robocop` available within the tutorial, and give this copy any other name (e.g. `robocop.arg`).
Now Robocop can be called with command lines like this:

1.) Simple check of single file with default configuration:

```
"%RobotPythonPath%/python.exe" -m robocop "⟨path⟩/mytestfile.robot"
```

2.) Check of single file with individual configuration taken from argument file:

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "⟨path⟩/robocop.arg" ↩
    ↪ "⟨path⟩/mytestfile.robot"
```

3.) Check of entire folder with individual configuration taken from argument file:

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "⟨path⟩/robocop.arg" "⟨folder⟩"
```

How to activate?

In Visual Studio Code the automated triggering of Robocop is activated per default within

```
RobotFramework\robotvscode\data\user-data\User\settings.json
```

with the following switches set to `true` :

```
"robot.lint.enabled": true,
"robot.lint.robocop.enabled": true,
```

Chapter 6

Logging

6.1 Background

The Robot Framework provides several log levels (also known as trace levels) to give users the possibility to control the amount of output in log files. The build in keyword `log` writes content to the log files. The optional parameter `level` of this keyword defines under which conditions (under which log level) the `log` keyword shall write the log message.

With command line parameter `--loglevel` a user defines this condition. If the log level of an executed `log` keyword matches to the log level defined for test execution, the log message is logged, otherwise not.

Available log levels in Robot Framework are: `ERROR` , `WARN` , `INFO` , `DEBUG` and `TRACE` . The default log level in Robot Framework is `INFO` . This level becomes active, if nothing else is specified by a user.

The log level `INFO` is also used by the Robot Framework itself, e.g. for the logging of informations about executed keywords. The impact is that every log file contains both the users output and the Robot Framework output. For larger tests this might cause unfavorable large log files.

Therefore the RobotFramework AIO provides an additional log level `USER` . Tests that are executed under this log level, do not contain the Robot Framework `INFO` messages any more (but for sure still contain errors and warnings).

Example code:

```
log    my log message    USER
```

Example execution:

```
python -m robot --loglevel USER -b logfile.log ./testfile.robot
```

6.2 Summary

In opposite to the Robot Framework (core), the RobotFramework AIO provides the following log levels to control the amount of logged content:

		log level in `log` keyword					
		ERROR	WARN	USER	INFO	DEBUG	TRACE
log level in command line `--loglevel`	ERROR	logged	----------	----------	----------	----------	----------
	WARN	logged	logged	----------	----------	----------	----------
	USER	logged	logged	logged	----------	----------	----------
	INFO	logged	logged	logged	logged	----------	----------
	DEBUG	logged	logged	logged	logged	logged	----------
	TRACE	logged	logged	logged	logged	logged	logged

The log level `USER` has been added to give users the possibility to log own content without all `INFO` messages logged by the Robot Framework (core).

Chapter 7

Library documentation

The following sections contain the documentation of additional libraries that are part of the Device Automation Service.

Device Automation Service bundle

Version 0.0.1.0 (from 03/2025)

Underlying Robot Framework (core)

Included libraries

MicroserviceBase
Version 0.1.1 (from 02.02.2023)
https://github.com/test-fullautomation/python-microservice-base
TODO

MicroserviceClewareSwitch
Version 0.1.2 (from 02.02.2023)
https://github.com/test-fullautomation/python-microservice-cleware-switch
TODO

7.1 MicroserviceBase

MicroserviceBase

v. 0.1.1

TODO

02.02.2023

Contents

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceBase**.

The sources of **MicroserviceBase** are available in GitHub.

Informations about how to install the **MicroserviceBase** can be found in the README.

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

__init__.py

biplist -- a library for reading and writing binary property list files.

Binary Property List (plist) files provide a faster and smaller serialization format for property lists on OS X. This is a library for generating binary plists which can be read by OS X, iOS, or other clients.

The API models the plistlib API, and will call through to plistlib when XML serialization or deserialization is required.

To generate plists with UID values, wrap the values with the Uid object. The value must be an int.

To generate plists with NSData/CFData values, wrap the values with the Data object. The value must be a string.

Date values can only be datetime.datetime objects.

The exceptions InvalidPlistException and NotBinaryPlistException may be thrown to indicate that the data cannot be serialized or deserialized as a binary plist.

Plist generation example:

> from biplist import * from datetime import datetime plist = {'aKey':'aValue', '0':1.322, 'now':datetime.now(), 'list':[1,2,3], 'tuple':('a','b','c') } try: writePlist(plist, "example.plist") except (InvalidPlistException, NotBinaryPlistException), e: print "Something bad happened:", e

Plist parsing example:

> from biplist import * try: plist = readPlist("example.plist") print plist except (InvalidPlistException, NotBinaryPlistException), e: print "Not a plist:", e

3.1 Function: readPlist

Raises NotBinaryPlistException, InvalidPlistException microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---wrapdataobject ==========

3.2 Function: writePlist

3.3 Function: readPlistFromString

3.4 Function: writePlistToString

3.5 Function: is_stream_binary_plist

3.6 Class: Uid

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Uid
```

Wrapper around integers for representing UID values. This is used in keyed archiving. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---data ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import Data
```

Wrapper around bytes to distinguish Data values. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---invalidplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import InvalidPlistException
```

Raised when the plist is incorrectly formatted. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---notbinaryplistexception ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import NotBinaryPlistException
```

Raised when a binary plist was expected but not encountered. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistreader ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistReader
```

3.6.1 Method: parse

3.6.2 Method: reset

3.6.3 Method: readRoot

3.6.4 Method: setCurrentOffsetToObjectNumber

3.6.5 Method: beginOffsetProtection

3.6.6 Method: endOffsetProtection

3.6.7 Method: readObject

3.6.8 Method: readContents

3.6.9 Method: readInteger

3.6.10 Method: readReal

3.6.11 Method: readRefs

3.6.12 Method: readArray

3.6.13 Method: readDict

3.6.14 Method: readAsciiString

3.6.15 Method: readUnicode

3.6.16 Method: readDate

3.6.17 Method: readData

3.6.18 Method: readUid

3.6.19 Method: getSizedInteger

Numbers of 8 bytes are signed integers when they refer to numbers, but unsigned otherwise. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---hashablewrapper ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import HashableWrapper
```

3.7 Class: BoolWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import BoolWrapper
```

3.8 Class: FloatWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import FloatWrapper
```

3.9 Class: StringWrapper

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import StringWrapper
```

3.9.1 Method: encodingMarker

3.10 Class: PlistWriter

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.biplist.__init__
↪  import PlistWriter
```

3.10.1 Method: reset

3.10.2 Method: positionOfObjectReference

If the given object has been written already, return its position in the offset table. Otherwise, return None. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeroot ----------

Strategy is: - write header - wrap root object so everything is hashable - compute size of objects which will be written - need to do this in order to know how large the object refs will be in the list/dict/set reference lists - write objects - keep objects in writtenReferences - keep positions of object references in referencePositions - write object references with the length computed previously - computer object reference length - write object reference positions - write trailer microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-beginrecursionprotection ----------

3.10.3 Method: endRecursionProtection

3.10.4 Method: wrapRoot

3.10.5 Method: incrementByteCount

3.10.6 Method: computeOffsets

3.10.7 Method: writeObjectReference

Tries to write an object reference, adding it to the references table. Does not write the actual object bytes or set the reference position. Returns a tuple of whether the object was a new reference (True if it was, False if it already was in the reference table) and the new output. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeobject ----------

Serializes the given object to the output. Returns output. If setReferencePosition is True, will set the position the object was written. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-writeoffsettable ----------

Writes all of the object reference offsets. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-binaryreal ----------

3.10.8 Method: binaryInt

3.10.9 Method: intSize

Returns the number of bytes necessary to store the given integer. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-biplist---init---plistwriter-realsize ----------

Chapter 4

badge.py

4.1 Function: badge_disk_icon

Chapter 5

colors.py

5.1 Function: isAColor

5.2 Function: parseColor

5.3 Class: Color

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Color
```

5.3.1 Method: to_rgb

5.4 Class: RGB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import RGB
```

5.4.1 Method: to_rgb

5.5 Class: HSL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HSL
```

5.5.1 Method: to_rgb

5.6 Class: HWB

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import HWB
```

5.6.1 Method: to_rgb

5.7 Class: CMYK

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import CMYK
```

5.7.1 Method: to_rgb

5.8 Class: Gray

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import Gray
```

5.8.1 Method: to_rgb

5.9 Class: ColorParser

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.colors
↪  import ColorParser
```

5.9.1 Method: skipws

5.9.2 Method: expect

5.9.3 Method: expectEnd

5.9.4 Method: getToken

5.9.5 Method: parseNumber

5.9.6 Method: parseColor

5.9.7 Method: parseRGB

5.9.8 Method: parseHSL

5.9.9 Method: parseHWB

5.9.10 Method: parseCMYK

5.9.11 Method: parseGray

5.9.12 Method: parseValue

5.9.13 Method: parseAngle

Chapter 6

core.py

6.1 Function: build_dmg

6.2 Class: DMGError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.dmgbuild.core
↪  import DMGError
```

Chapter 7

buddy.py

7.1 Class: BuddyError

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import BuddyError
```

7.2 Class: Block

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Block
```

7.2.1 Method: close

7.2.2 Method: flush

7.2.3 Method: invalidate

7.2.4 Method: zero_fill

7.2.5 Method: tell

7.2.6 Method: seek

7.2.7 Method: read

7.2.8 Method: write

7.2.9 Method: insert

7.2.10 Method: delete

7.3 Class: Allocator

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.buddy
↪  import Allocator
```

7.3.1 Method: open

7.3.2 Method: close

7.3.3 Method: flush

7.3.4 Method: read

Read data at ‘offset', or raise an exception. ‘size_or_format' may either be a byte count, in which case we return raw data, or a format string for ‘struct.unpack', in which case we work out the size and unpack the data before returning it. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-write -------
---

Write data at ‘offset', or raise an exception. ‘data_or_format' may either be the data to write, or a format string for ‘struct.pack', in which case we pack the additional arguments and write the resulting data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-get-block ----------
---

7.3.5 Method: allocate

Allocate or reallocate a block such that it has space for at least ‘bytes' bytes. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-buddy-allocator-release ----------
---

7.3.6 Method: iterkeys

7.3.7 Method: keys

Chapter 8

store.py

8.1 Class: ILocCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import ILocCodec
```

8.1.1 Method: encode

8.1.2 Method: decode

8.2 Class: PlistCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import PlistCodec
```

8.2.1 Method: encode

8.2.2 Method: decode

8.3 Class: BookmarkCodec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import BookmarkCodec
```

8.3.1 Method: encode

8.3.2 Method: decode

8.4 Class: DSStoreEntry

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import DSStoreEntry
```

Holds the data from an entry in a `.DS_Store` file. Note that this is not meant to represent the entry itself---i.e. if you change the type or value, your changes will *not* be reflected in the underlying file.

If you want to make a change, you should either use the `DSStore` object's `DSStore.insert` method (which will replace a key if it already exists), or the mapping access mode for `DSStore` (often simpler anyway). microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-read ----------

Read a `.DS_Store` entry from the containing Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-byte-length ----------

Compute the length of this entry, in bytes microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstoreentry-write ----------

Write this entry to the specified Block microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.ds_store.store
↪  import DSStore
```

Python interface to a `.DS_Store` file. Works by manipulating the file on the disk---so this code will work with `.DS_Store` files for *very* large directories.

A `DSStore` object can be used as if it was a mapping, e.g.:

```
d['foobar.dat']['Iloc']
```

will fetch the "Iloc" record for "foobar.dat", or raise `KeyError` if there is no such record. If used in this manner, the `DSStore` object will return (type, value) tuples, unless the type is "blob" and the module knows how to decode it.

Currently, we know how to decode "Iloc", "bwsp", "lsvp", "lsvP" and "icvp" blobs. "Iloc" decodes to an (x, y) tuple, while the others are all decoded using `biplist`.

Assignment also works, e.g.:

```
d['foobar.dat']['note'] = ('ustr', u'Hello World!')
```

as does deletion with `del`:

```
del d['foobar.dat']['note']
```

This is usually going to be the most convenient interface, though occasionally (for instance when creating a new `.DS_Store` file) you may wish to drop down to using `DSStoreEntry` objects directly. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-open ----------

Open a `.DS_Store` file; pass either a Python file object, or a filename in the `file_or_name` argument and a file access mode in the `mode` argument. If you are creating a new file using the "w" or "w+" modes, you may also specify a list of entries with which to initialise the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-flush ----------

Flush any dirty data back to the file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-close ----------

Flush dirty data and close the underlying file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-insert ----------

Insert `entry` (which should be a `DSStoreEntry`) into the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-delete ----------

Delete an item, identified by `filename` and `code` from the B-Tree. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-ds-store-store-dsstore-find ----------

Returns a generator that will iterate over matching entries in the B-Tree.

Chapter 9

alias.py

9.1 Function: encode_utf8

9.2 Function: decode_utf8

9.3 Class: AppleShareInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import AppleShareInfo
```

9.4 Class: VolumeInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import VolumeInfo
```

9.4.1 Method: filesystem_type

9.5 Class: TargetInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import TargetInfo
```

9.6 Class: Alias

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.alias
↪  import Alias
```

9.6.1 Method: from_bytes

Construct an `Alias` object given binary Alias data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-for-file ----------

Create an `Alias` that points at the specified file. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-alias-alias-to-bytes ----------

Returns the binary representation for this `Alias`.

Chapter 10

bookmark.py

10.1 Function: iteritems

10.2 Class: Data

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Data
```

10.3 Class: URL

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import URL
```

10.3.1 Method: absolute

Return an absolute URL. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark ==========

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.bookmar
↪  import Bookmark
```

10.3.2 Method: from_bytes

Create a `Bookmark` given byte data. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-get ----------

Lookup the value for a given key, returning a default if not present. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-to-bytes ----------

Convert this `Bookmark` to a byte representation. microservicebase-microservicemanagergui-node-modules-dmg-builder-vendor-mac-alias-bookmark-bookmark-for-file ----------

Construct a `Bookmark` for a given file.

Chapter 11

osx.py

11.1 Function: getattrlist

11.2 Function: fgetattrlist

11.3 Function: statfs

11.4 Function: fstatfs

11.5 Class: attrlist

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrlist
```

11.6 Class: attribute_set_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attribute_set_t
```

11.7 Class: fsobj_id_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsobj_id_t
```

11.8 Class: timespec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import timespec
```

11.9 Class: attrreference_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import attrreference_t
```

11.10 Class: fsid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import fsid_t
```

11.11 Class: guid_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import guid_t
```

11.12 Class: kauth_ace

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_ace
```

11.13 Class: kauth_acl

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_acl
```

11.14 Class: kauth_filesec

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import kauth_filesec
```

11.15 Class: diskextent

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import diskextent
```

11.16 Class: Point

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import Point
```

11.17 Class: Rect

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import Rect
```

11.18 Class: FileInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import FileInfo
```

11.19 Class: FolderInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import FolderInfo
```

11.20 Class: FinderInfo

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import FinderInfo
```

11.21 Class: vol_capabilities_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_capabilities_attr_t
```

11.22 Class: vol_attributes_attr_t

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import vol_attributes_attr_t
```

11.23 Class: struct_statfs

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.osx
↪  import struct_statfs
```

Chapter 12

utils.py

12.1 Class: UTC

Imported by:

```
from
↪  MicroserviceBase.MicroserviceManagerGUI.node_modules.dmg-builder.vendor.mac_alias.utils
↪  import UTC
```

12.1.1 Method: utcoffset

12.1.2 Method: dst

12.1.3 Method: tzname

Chapter 13

ServiceBase.py

13.1 Class: ResultType

Imported by:

```
from MicroserviceBase.ServiceRegistry.ServiceBase import ResultType
```

Result Types.

13.2 Class: ResponseMessage

Imported by:

```
from MicroserviceBase.ServiceRegistry.ServiceBase import ResponseMessage
```

Response message class

13.2.1 Method: get_json

Convert the response message to JSON format.

Returns:

/ *Type*: str /
The response message in JSON format.

13.2.2 Method: get_data

Retrieve the result data as a string.

Returns:

/ *Type*: str /
The result data as a string.

13.3 Class: ServiceBase

Imported by:

```
from MicroserviceBase.ServiceRegistry.ServiceBase import ServiceBase
```

13.3.1 Method: parse_arguments

Parse basic service arguments from the command line.

Arguments:

- `cmd_args`
  / *Condition*: required / *Type*: list /
  Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed arguments.

13.3.2 Method: parse_spec_arguments

Parse specific arguments for each customized service.

This method should be overridden by subclasses to handle service-specific arguments.

Arguments:

- `cmd_args`
  / *Condition*: required / *Type*: list /
  Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed specific arguments.

13.3.3 Method: close

Close the service connection.

Returns:

(*no returns*)

13.3.4 Method: create_request_data

Create request data for a given method name and arguments.

Arguments:

- `method_name`
  / *Condition*: required / *Type*: str /
  The name of the method for which the request is being created.
- `args`
  / *Condition*: required / *Type*: list /
  The list of arguments for the method.

Returns:

/ *Type*: dict /
A dictionary containing the method name and arguments.

13.3.5 Method: connect_broker

Establish a connection to the broker.

Arguments:

- `**kwargs`
  / *Condition*: optional / *Type*: dict /
  Additional keyword arguments for broker connection parameters.

Returns:

(*no returns*)

13.3.6 Method: serve

Call to start service serving.

Returns:

(*no returns*)

13.3.7 Method: register_service

Register a service to the ServiceRegistry.

Returns:

(*no returns*)

13.3.8 Method: unregister_service

Unregister a service from the ServiceRegistry.

Returns:

(*no returns*)

13.3.9 Method: request_service

Send a service request to a specific exchange with a given routing key.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  The data for the service request.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  The name of the exchange to send the request to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the request.

Returns:

(*no returns*)

13.3.10 Method: get_svc_api_methods_dict

Retrieve all service API methods provided by the service (methods starting with the prefix 'svc_api').

Returns:

/ *Type*: dict /
A dictionary containing the names and references of all service API methods.

13.3.11 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from the docstrings of the methods.

Arguments:

- `methods_dict`
  / *Condition*: required / *Type*: dict /
  A dictionary containing the names and references of all service API methods.

Returns:

/ *Type*: dict /
A dictionary containing the information of all service APIs extracted from their docstrings.

13.3.12 Method: parse_docstring

Parse function's docstring to get arguments and return information.

Arguments:

- `docstring`
  / *Condition*: required / *Type*: str /
  Function's docstring.

Returns:

/ *Type*: str /
Payloads of the waiting signal if received.

13.3.13 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

13.3.14 Method: svc_api_get_gui_files

13.3.15 Method: is_specific_request

Check if the request is a specific request.

Arguments:

- `request`
  / *Condition*: required / *Type*: object /
  The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

13.3.16 Method: on_specific_request

Handle the event when a specific request is received.
Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `props`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: dict /
  The body of the message as a dictionary.

Returns:
(*no returns*)

13.3.17 Method: on_request

Handle an incoming request.
Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `props`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: bytes /
  The body of the message as bytes.

Returns:
(*no returns*)

Chapter 14

ServiceRegistry.py

14.1 Function: signal_handler

Handle signals from the operating system.

Arguments:

- `sig`
  / *Condition*: required / *Type*: int /
  The signal number received from the OS.
- `frame`
  / *Condition*: required / *Type*: frame object /
  The current stack frame.
- `obj`
  / *Condition*: required / *Type*: object /
  The object that is handling the signal.

Returns:

(*no returns*)

14.2 Class: ServiceRegistry

Imported by:

```
from MicroserviceBase.ServiceRegistry.ServiceRegistry import ServiceRegistry
```

Manage information for all services connected to the broker it is connected to.

This class handles the registration, updates, and retrieval of service information, ensuring that all services interacting with the broker are properly managed and monitored.

14.2.1 Method: receive_services_information

Run in a thread to listen for any changes from the services.

Returns:

(*no returns*)

14.2.2 Method: handle_update

Handle the event when service information is updated.

Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `properties`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: bytes /
  The body of the message as bytes.

Returns:
(*no returns*)

14.2.3 Method: notify_updates

Notify updates to the realtime update channel.
Returns:
(*no returns*)

14.2.4 Method: svc_api_get_services_info

Retrieve information of all services connected to the broker that the Service Registry is connected to.
Returns:

/ *Type*: dict /
A dictionary containing information of all connected services.

14.2.5 Method: svc_api_get_realtime_update_exchange

Retrieve the exchange name of the realtime update exchange.
Returns:

/ *Type*: str /
The name of the realtime update exchange.

14.2.6 Method: svc_api_update_alias_conf

Update the alias configuration information.
Arguments:

- `alias_string`
  / *Condition*: required / *Type*: str /
  The alias configuration string to be updated.

Returns:
(*no returns*)

14.2.7 Method: svc_api_get_alias_conf

Retrieve the alias configuration string in JSON format.

Returns:

/ *Type*: str /
The alias configuration string in JSON format.

14.2.8 Method: is_specific_request

Check if the request is a specific request.

Arguments:

- `request`
  / *Condition*: required / *Type*: object /
  The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

14.2.9 Method: on_specific_request

Handle the event when a specific request is received.

Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `props`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: dict /
  The body of the message as a dictionary.

Returns:
(*no returns*)

Chapter 15

Appendix

About this package:

Table 15.1: Package setup

Setup parameter	Value
Name	MicroserviceBase
Version	0.1.1
Date	02.02.2023
Description	TODO
Package URL	python-microservice-base
Author	TODO
Email	TODO
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 16

History

0.1.0	02/2023
Initial version	

MicroserviceBase.pdf
Created at 25.03.2025 - 10:52:40
by GenPackageDoc v. 0.16.0

7.2 MicroserviceClewareSwitch

MicroserviceClewareSwitch

v. 0.1.2

TODO

02.02.2023

Contents

Chapter 1

Introduction

TODO: Add introduction of **MicroserviceClewareSwitch**.

The sources of **MicroserviceClewareSwitch** are available in GitHub.

Informations about how to install the **MicroserviceClewareSwitch** can be found in the README.

Chapter 2

Description

2.1 ToDo

Description

2.1.1 ToDo ToDo

Description

ToDo ToDo ToDo

Description

Chapter 3

ClewareAccessHelper.py

3.1 Class: ClewareAccessHelper

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelper import ClewareAccessHelper
```

ClewareAccessHelper class acts as a proxy class in Proxy pattern and forward the request to the real helper depend on platform. microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-load-config ----------

Load the user config port name for USB access. Returns: None microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-get-config ----------

Get Cleware ports' config. Returns: Dictionary of ports' setting. microserviceclewareswitch-clewareaccesshelper-clewareaccesshelper-set-config ----------

Save Cleware ports' config to file. Args: config_json: data to be saved.

Returns: res: saved result.

Chapter 4

ClewareAccessHelperAbs.py

4.1 Class: ClewareAccessHelperAbs

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperAbs import ClewareAccessHelperAbs
```

ClewareAccessHelperAbs acts as a abstract class for the real Helper in Proxy Pattern. microserviceclewareswitch-clewareaccesshelperabs-clewareaccesshelperabs-init-cleware ----------

4.1.1 Method: open_cleware

4.1.2 Method: close_cleware

4.1.3 Method: set_value

4.1.4 Method: get_value

4.1.5 Method: set_switch

4.1.6 Method: get_switch

4.1.7 Method: get_handle

4.1.8 Method: get_version

4.1.9 Method: get_usb_type

4.1.10 Method: get_usb_type

4.1.11 Method: get_serial_number

4.1.12 Method: valid_ser_number

4.1.13 Method: get_hw_version

4.1.14 Method: is_ampel

4.1.15 Method: iox

Chapter 5

ClewareAccessHelperLinux.py

5.1 Class: ClewareAccessHelperLinux

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperLinux import
↪  ClewareAccessHelperLinux
```

ClewareAccessHelperLinux acts as the real object on Linux platform in Proxy Pattern

- This is the wrapper class for the functions provided by C/C++ source for Linux.
- The C/C++ source for Linux is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN
- The C/C++ source for Linux is modified by Cuong Nguyen (RBVH/ENG22) for support python to load dynamic library on Linux.

5.1.1 Method: init_cleware

5.1.2 Method: open_cleware

5.1.3 Method: close_cleware

5.1.4 Method: get_handle

5.1.5 Method: set_value

5.1.6 Method: get_value

5.1.7 Method: set_switch

5.1.8 Method: get_switch

5.1.9 Method: get_version

5.1.10 Method: get_usb_type

5.1.11 Method: get_serial_number

5.1.12 Method: valid_ser_number

5.1.13 Method: get_hw_version

5.1.14 Method: is_ampel

5.1.15 Method: iox

5.1.16 Method: get_all_sw_state

Chapter 6

ClewareAccessHelperWindows.py

6.1 Class: ClewareAccessHelperWindows

Imported by:

```
from MicroserviceClewareSwitch.ClewareAccessHelperWindows import
↪  ClewareAccessHelperWindows
```

ClewareAccessHelperWindows acts as the real object on Windows platform in Proxy Pattern

- This is the wrapper class for the functions provided by USBaccess.dll.
- The USBaccess.dll is provided by Cleware manufacturer at : http://www.cleware-shop.de/Downloads_EN

6.1.1 Method: init_cleware

6.1.2 Method: open_cleware

6.1.3 Method: close_cleware

6.1.4 Method: get_handle

6.1.5 Method: set_value

6.1.6 Method: get_value

6.1.7 Method: set_switch

6.1.8 Method: get_switch

6.1.9 Method: get_version

6.1.10 Method: get_usb_type

6.1.11 Method: get_serial_number

6.1.12 Method: valid_ser_number

6.1.13 Method: get_hw_version

6.1.14 Method: is_ampel

6.1.15 Method: iox

Chapter 7

ServiceBase.py

7.1 Class: ResultType

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ResultType
```

Result Types.

7.2 Class: ResponseMessage

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ResponseMessage
```

Response message class

7.2.1 Method: get_json

Convert the response message to JSON format.

Returns:

/ *Type*: str /
The response message in JSON format.

7.2.2 Method: get_data

Retrieve the result data as a string.

Returns:

/ *Type*: str /
The result data as a string.

7.3 Class: ServiceBase

Imported by:

```
from MicroserviceClewareSwitch.ServiceBase import ServiceBase
```

7.3.1 Method: parse_arguments

Parse basic service arguments from the command line.

Arguments:

- `cmd_args`
  / *Condition*: required / *Type*: list /
  Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed arguments.

7.3.2 Method: parse_spec_arguments

Parse specific arguments for each customized service.

This method should be overridden by subclasses to handle service-specific arguments.

Arguments:

- `cmd_args`
  / *Condition*: required / *Type*: list /
  Command-line arguments to be parsed.

Returns:

/ *Type*: dict /
A dictionary containing the parsed specific arguments.

7.3.3 Method: close

Close the service connection.

Returns:

(*no returns*)

7.3.4 Method: create_request_data

Create request data for a given method name and arguments.

Arguments:

- `method_name`
  / *Condition*: required / *Type*: str /
  The name of the method for which the request is being created.
- `args`
  / *Condition*: required / *Type*: list /
  The list of arguments for the method.

Returns:

/ *Type*: dict /
A dictionary containing the method name and arguments.

7.3.5 Method: connect_broker

Establish a connection to the broker.

Arguments:

- `**kwargs`
  / *Condition*: optional / *Type*: dict /
  Additional keyword arguments for broker connection parameters.

Returns:

(*no returns*)

7.3.6 Method: serve

Call to start service serving.

Returns:

(*no returns*)

7.3.7 Method: register_service

Register a service to the ServiceRegistry.

Returns:

(*no returns*)

7.3.8 Method: unregister_service

Unregister a service from the ServiceRegistry.

Returns:

(*no returns*)

7.3.9 Method: request_service

Send a service request to a specific exchange with a given routing key.

Arguments:

- `request_data`
  / *Condition*: required / *Type*: dict /
  The data for the service request.
- `exchange_name`
  / *Condition*: required / *Type*: str /
  The name of the exchange to send the request to.
- `routing_key`
  / *Condition*: required / *Type*: str /
  The routing key for the request.

Returns:

(*no returns*)

7.3.10 Method: get_svc_api_methods_dict

Retrieve all service API methods provided by the service (methods starting with the prefix 'svc_api').

Returns:

/ *Type*: dict /
A dictionary containing the names and references of all service API methods.

7.3.11 Method: get_svc_api_methods_info_dict

Retrieve information for all service APIs from the docstrings of the methods.

Arguments:

- `methods_dict`
  / *Condition*: required / *Type*: dict /
  A dictionary containing the names and references of all service API methods.

Returns:

/ *Type*: dict /
A dictionary containing the information of all service APIs extracted from their docstrings.

7.3.12 Method: parse_docstring

Parse function's docstring to get arguments and return information.

Arguments:

- `docstring`
  / *Condition*: required / *Type*: str /
  Function's docstring.

Returns:

/ *Type*: str /
Payloads of the waiting signal if received.

7.3.13 Method: svc_api_get_version

Get the service version.

Returns:

/ *Type*: str /
Version of the service.

7.3.14 Method: svc_api_get_gui_files

7.3.15 Method: svc_api_get_sample

7.3.16 Method: is_specific_request

Check if the request is a specific request.

Arguments:

- `request`
  / *Condition*: required / *Type*: object /
  The request object to be checked.

Returns:

/ *Type*: bool /
True if the request is a specific request, otherwise False.

7.3.17 Method: on_specific_request

Handle the event when a specific request is received.

Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `props`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: dict /
  The body of the message as a dictionary.

Returns:

(*no returns*)

7.3.18 Method: on_request

Handle an incoming request.

Arguments:

- `ch`
  / *Condition*: required / *Type*: pika.channel.Channel /
  The channel object from the pika library.
- `method`
  / *Condition*: required / *Type*: pika.spec.Basic.Deliver /
  The method object containing delivery information from the pika library.
- `props`
  / *Condition*: required / *Type*: pika.spec.BasicProperties /
  The properties of the message from the pika library.
- `body`
  / *Condition*: required / *Type*: bytes /
  The body of the message as bytes.

Returns:

(*no returns*)

Chapter 8

ServiceCleware.py

8.1 Function: signal_handler

Handle signals from the operating system.
Arguments:

- `sig`
  / *Condition*: required / *Type*: int /
  The signal number received from the OS.
- `frame`
  / *Condition*: required / *Type*: frame object /
  The current stack frame.
- `obj`
  / *Condition*: required / *Type*: object /
  The object that is handling the signal.

Returns:
(*no returns*)

8.2 Class: ServiceCleware

Imported by:

```
from MicroserviceClewareSwitch.ServiceCleware import ServiceCleware
```

Service for controlling Cleware devices.
This class extends ServiceBase to provide functionalities specific to managing and controlling Cleware devices.

8.2.1 Method: svc_api_get_all_devices_state

Retrieve the state of all Cleware devices.
Returns:

/ *Type*: dict /
A dictionary containing the states of all Cleware devices.

8.2.2 Method: svc_api_set_switch

Set state for a Cleware device's switch.

Arguments:

- `device_no`
  / *Condition*: required / *Type*: str /
  Cleware device's number.
- `switch_id`
  / *Condition*: required / *Type*: str /
  Switch number to turn on or off.
- `state`
  / *Condition*: required / *Type*: str /
  State of swith to set (on/off).

Returns:

/ *Type*: int /
Return ret code, 1 for succeed, 0 for failure.

8.2.3 Method: notify_updates

Notify updates to the realtime update channel for Cleware devices.

Returns:

(*no returns*)

Chapter 9

ServiceLogger.py

9.1 Class: ServiceLogger

Imported by:

```
from MicroserviceClewareSwitch.ServiceLogger import ServiceLogger
```

Logger class. microserviceclewareswitch-servicelogger-servicelogger-set-logger ----------

9.1.1 Method: get_config_file

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

9.1.2 Method: log

9.1.3 Method: log_info

9.1.4 Method: log_debug

9.1.5 Method: log_warning

9.1.6 Method: log_error

9.1.7 Method: log_critical

Chapter 10

Utils.py

10.1 Class: Singleton

Imported by:

```
from MicroserviceClewareSwitch.Utils import Singleton
```

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s) Used when class has too few public methods, so be sure it's really worth it.

> This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

10.2 Class: Utils

Imported by:

```
from MicroserviceClewareSwitch.Utils import Utils
```

Class to implement utilities for supporting development. microserviceclewareswitch-utils-utils-get-all-descendant-classes ----------

Get all descendant classes of a class Args: cls: Input class for finding descendants.

Returns: Array of descendant classes.

10.2.1 Method: get_all_sub_classes

Get all children classes of a class Args: cls: Input class for finding children.

Returns: Array of children classes.

10.2.2 Method: caller_name

Get a name of a caller in the format module.class.method Args: skip: specifies how many levels of stack to skip while getting caller name. skip=1 means "who calls me", skip=2 "who calls my caller" etc.

Returns: An empty string is returned if skipped levels exceed stack height

10.2.3 Method: load_library

Load native library depend on the calling convention. Args: path: library path. is_stdcall: determine if the library's calling convention is stdcall or cdecl.

Returns: Loaded library object.

10.2.4 Method: is_ascii_or_unicode

Check if the string is ascii or unicode Args: str_check: string for checking codecs: encoding type list

Returns: True : if checked string is ascii or unicode False : if checked string is not ascii or unicode

10.2.5 Method: get_registry_parameters

Get service's parameters which stored in registry. Args: service_name: Name of the service. key_name: Name of register key.

Returns: Value of service's parameters

10.2.6 Method: create_registry_parameters

Create service's parameters and store it to registry. Args: service_name: Name of the service. key_name: Name of register key. parameters: Parameters to be stored.

Returns: None

10.3 Class: Job

Imported by:

```
from MicroserviceClewareSwitch.Utils import Job
```

10.3.1 Method: stop

10.3.2 Method: run

10.4 Class: ResultType

Imported by:

```
from MicroserviceClewareSwitch.Utils import ResultType
```

Result Types. microserviceclewareswitch-utils-responsemessage ==========

Imported by:

```
from MicroserviceClewareSwitch.Utils import ResponseMessage
```

Response message class microserviceclewareswitch-utils-responsemessage-get-json ----------

Convert response message to json Returns: Response message in json format microserviceclewareswitch-utils-responsemessage-get-data ----------

Get string data result Returns: String result microserviceclewareswitch-utils-responsemessage-create-from-string ----------

Chapter 11

Appendix

About this package:

Table 11.1: Package setup

Setup parameter	Value
Name	MicroserviceClewareSwitch
Version	0.1.2
Date	02.02.2023
Description	TODO
Package URL	python-microservice-cleware-switch
Author	TODO
Email	TODO
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 12

History

0.1.0	02/2023
Initial version	

MicroserviceClewareSwitch.pdf
Created at 25.03.2025 - 10:53:05
by GenPackageDoc v. 0.16.0

Chapter 8

Appendix

8.1 Installed Python Modules

This chapter contains a list of installed Python modules together with their version numbers.

Based on: Python 3.9.16 (main, Dec 21 2022, 04:16:27) [MSC v.1929 64 bit (AMD64)]

Identified 296 packages.

aenum : 3.1.15
aiodns : 3.2.0
aiofiles : 24.1.0
aiohappyeyeballs : 2.5.0
aiohttp : 3.11.13
aiosignal : 1.3.2
alabaster : 0.7.16
altgraph : 0.17.4
anyio : 4.9.0
appdirs : 1.4.4
Appium-Python-Client : 3.2.1
argon2-cffi : 23.1.0
argon2-cffi-bindings : 21.2.0
argparse-addons : 0.12.0
arrow : 1.3.0
asciidoxy : 0.5.2
astroid : 3.3.8
asttokens : 3.0.0
async-lru : 2.0.5
async-timeout : 5.0.1
attrs : 25.1.0
babel : 2.17.0
bcrypt : 4.3.0
beautifulsoup4 : 4.13.3
bitstruct : 8.20.0
black : 25.1.0
bleach : 6.2.0
bokeh : 3.4.3
cchardet : 2.1.7
certifi : 2025.1.31
cffi : 1.17.1
cfgv : 3.4.0
chardet : 5.2.0
charset-normalizer : 3.4.1
click : 8.1.8
colorama : 0.4.6
comm : 0.2.2
configparser : 7.1.0
contourpy : 1.3.0
coverage : 7.6.12
cryptography : 44.0.2
cycler : 0.12.1
debugpy : 1.8.13
decorator : 5.2.1
defusedxml : 0.7.1
deprecation : 2.1.0
dill : 0.3.9
distlib : 0.3.9
docopt : 0.6.2
docutils : 0.21.2
doipclient : 1.1.4
dotdict : 0.1
dotmap : 1.3.30
doxypypy : 0.8.8.7
EasyProcess : 1.1
elementpath : 4.8.0
entrypoint2 : 1.1
et_xmlfile : 2.0.0
exceptiongroup : 1.2.2
executing : 2.2.0
fastjsonschema : 2.21.1
filelock : 3.17.0
fonttools : 4.56.0
fqdn : 1.5.1
frozenlist : 1.5.0
func_timeout : 4.3.5
GenPackageDoc : 0.41.1
h11 : 0.14.0
httpcore : 1.0.7
httpserver : 1.1.0
httpx : 0.28.1
identify : 2.6.8
idna : 3.10
imageio : 2.37.0
imagesize : 1.4.1
importlib_metadata : 8.6.1
importlib_resources : 6.5.2
imutils : 0.5.4
iniconfig : 2.0.0
install-requires : 0.3.0
ipykernel : 6.29.5
ipython : 8.18.1
ipywidgets : 8.1.5
isoduration : 20.11.0
isort : 6.0.1
jedi : 0.19.2
Jinja2 : 3.1.6
joblib : 1.4.2
json5 : 0.10.0
jsonpointer : 3.0.0
JsonPreprocessor : 0.8.4
jsonschema : 4.23.0
jsonschema-specifications : 2024.10.1
jupyter-events : 0.12.0
jupyter-lsp : 2.2.5
jupyter_client : 8.6.3
jupyter_core : 5.7.2
jupyter_server : 2.15.0
jupyter_server_terminals : 0.5.3
jupyterlab : 4.3.6
jupyterlab_pygments : 0.3.0
jupyterlab_server : 2.27.3
jupyterlab_widgets : 3.0.13
kitchen : 1.2.6
kiwisolver : 1.4.7
lazy_loader : 0.4
lunr : 0.8.0
lxml : 5.3.1
Mako : 1.3.9
markdown-it-py : 3.0.0
markdownify : 1.1.0

MarkupSafe : 3.0.2
matplotlib : 3.9.4
matplotlib-inline : 0.1.7
mccabe : 0.7.0
mdurl : 0.1.2
MicroserviceBase : 0.1.1
MicroserviceClewareSwitch : 0.1.2
mistune : 3.1.2
mobly : 1.12.2
msl-loadlib : 0.10.0
mss : 10.0.0
multidict : 6.1.0
mypy-extensions : 1.0.0
mysqlclient : 2.2.7
nbclient : 0.10.2
nbconvert : 7.16.6
nbformat : 5.10.4
nest-asyncio : 1.6.0
networkx : 3.2.1
nodeenv : 1.9.1
notebook : 7.3.3
notebook_shim : 0.2.4
numpy : 2.0.2
odxtools : 9.5.0
opencv-python-headless : 4.11.0.86
openpyxl : 3.1.5
ordereddict : 1.1
orjson : 3.10.15
outcome : 1.3.0.post0
overrides : 7.7.0
packaging : 24.2
paho-mqtt : 2.1.0
pandas : 2.2.3
pandocfilters : 1.5.1
paramiko : 3.5.1
parso : 0.8.4
path : 17.1.0
path.py : 12.5.0
pathspec : 0.12.1
pefile : 2023.2.7
pickleDB : 1.3.2
pika : 1.3.2
pillow : 11.1.0
platformdirs : 4.3.6
pluggy : 1.5.0
ply : 3.11
portpicker : 1.6.0
pre_commit : 4.1.0
pre_commit_hooks : 5.0.0
prometheus_client : 0.21.1
prompt_toolkit : 3.0.50
propcache : 0.3.0
psutil : 7.0.0
pure_eval : 0.2.3
pycares : 4.5.0
pycparser : 2.22
pydotdict : 3.3.11
pyfranca : 0.4.1
Pygments : 2.19.1
pyinstaller : 6.12.0
pyinstaller-hooks-contrib : 2025.1
pylint : 3.3.4
PyNaCl : 1.5.0
pyodbc : 5.2.0
pypandoc_binary : 1.13
pyparsing : 3.2.1
PyQt5 : 5.15.11
PyQt5-Qt5 : 5.15.2
PyQt5_sip : 12.17.0
pyrsistent : 0.20.0
pyscreenshot : 3.1
pyserial : 3.4
pyshark : 0.6
PySocks : 1.7.1
pyspnego : 0.11.2
pytest : 8.3.5
pytest-mock : 3.14.0
PyTestLog2DB : 0.3.4
python-can : 4.5.0
python-dateutil : 2.9.0.post0
python-json-logger : 3.3.0
PythonExtensionsCollection : 0.16.0
pyttsx : 1.1
pytz : 2025.1
pywin32 : 308
pywin32-ctypes : 0.2.3
pywinpty : 2.0.15
PyYAML : 6.0.2
pyzmq : 26.2.1
QLed : 1.3.1
QtAwesome : 1.4.0
QtPy : 2.4.3
referencing : 0.36.2
regex : 2024.11.6
requests : 2.32.3
requests-kerberos : 0.15.0
rfc3339-validator : 0.1.4
rfc3986-validator : 0.1.1
rich : 13.9.4
rich-click : 1.8.5
robotframework : 6.1
robotframework-appiumlibrary : 2.1.0
robotframework-dbus : 0.1.3
robotframework-dependencylibrary : 4.0.1
robotframework-documentation : 0.36.0
robotframework-doip : 0.1.5
robotframework-excellib : 2.0.1
robotframework-extensions-collection : 0.10.0
robotframework-ftplibrary : 1.9
robotframework-imagecompare : 0.2.0
robotframework-listenerlibrary : 1.0.3
robotframework-mqttlibrary : 0.7.1.post3
robotframework-prometheus : 0.8.0
robotframework-pythonlibcore : 4.4.1
robotframework-qconnect-base : 1.1.3
robotframework-qconnect-winapp : 1.0.3
robotframework-requests : 0.9.7
robotframework-robocop : 5.8.1
robotframework-robotlog2db : 1.5.4
robotframework-robotlog2rqm : 1.4.2
robotframework-seriallibrary : 0.4.3
robotframework-sshlibrary : 3.8.0
robotframework-testsuitesmanagement : 0.7.12
robotframework-tidy : 4.16.0
RobotFramework_UDS : 0.1.12
robotkernel : 1.6
rpds-py : 0.23.1
ruamel.yaml : 0.18.10
ruamel.yaml.clib : 0.2.12
ruff : 0.9.9
schedule : 1.2.2
scikit-image : 0.24.0
scikit-learn : 1.6.1
scipy : 1.13.1
scp : 0.15.0
selenium : 4.16.0
Send2Trash : 1.8.3
six : 1.17.0
sniffio : 1.3.1
snowballstemmer : 2.2.0
sortedcontainers : 2.4.0
soupsieve : 2.6
Sphinx : 7.4.7
sphinxcontrib-applehelp : 2.0.0
sphinxcontrib-devhelp : 2.0.0
sphinxcontrib-htmlhelp : 2.1.0
sphinxcontrib-jsmath : 1.0.1
sphinxcontrib-qthelp : 2.0.0
sphinxcontrib-serializinghtml : 2.0.0
sspilib : 0.2.0
stack-data : 0.6.3
termcolor : 2.5.0
terminado : 0.18.1
TestResultDBAccess : 0.1.4
threadpoolctl : 3.5.0
tifffile : 2024.8.30
tinycss2 : 1.4.0
tk : 0.1.0
toml : 0.10.2
tomli : 2.2.1
tomlkit : 0.13.2
tornado : 6.4.2
traitlets : 5.14.3
trio : 0.29.0
trio-websocket : 0.12.2
types-python-dateutil : 2.9.0.20241206
typing_extensions : 4.12.2
tzdata : 2025.1
udsoncan : 1.23.2
uri-template : 1.3.0
urllib3 : 2.3.0
virtualenv : 20.29.2
Wave : 0.0.2
wcwidth : 0.2.13
webcolors : 24.11.1
webencodings : 0.5.1
websocket-client : 1.8.0
widgetsnbextension : 4.0.13

wrapt : 1.17.2
wsproto : 1.2.0
xmlschema : 3.4.3
xmltodict : 0.14.2
xyzservices : 2025.1.0
yarl : 1.18.3
zipp : 3.21.0

Additionally required Python packages can be installed in this way:

1. Windows:

```
"%RobotPythonPath%/python.exe" -m pip install --proxy <proxy address> <packagename>
```

2. Linux:

```
"${RobotPythonPath}/python3" -m pip install --proxy <proxy address> <packagename>
```

The full path and name of the Python interpreter is required in these command lines because the **RobotFramework AIO** installer does not modify the environment of the computer (except the setup of some environment variables).

The proxy address is an option and depends on the conditions under which your company grants the access to the internet.